

FULL STACK ONLINE COMPLAINT REGISTRATION(MERN)

1. Introduction

Project Title: ONLINE COMPLAINT REGISTRATION(FULL-STACK MERN)

Team Members:

1.BODDUPALLI VENKATA RAJYA LAKSHMI - Full Stack Developer (Frontend, Backend, Database,Testing & Documentation)

Project Description:

The **Online Complaint Registration** is a web-based application developed using the **MERN Stack (MongoDB, Express.js, React.js, and Node.js)**. The system is designed to simplify and automate the process of registering, managing, assigning, tracking, and resolving complaints through a centralized platform.

Traditionally, complaint handling involves manual paperwork, delayed communication, and difficulty in tracking complaint status. This project addresses these challenges by providing an efficient digital platform where users can submit complaints, monitor their progress, and communicate with the concerned authorities.

The application supports three different user roles: Ordinary User, Agent, and Admin. Ordinary users can register, log in, submit complaints with supporting images, track complaint status, update their profiles, and provide feedback after complaint resolution. Agents can view assigned complaints, update complaint status, upload resolution images, and resolve complaints. Administrators can manage users, approve agents, assign complaints, monitor complaint statistics through dashboards, and oversee the complete complaint management process.

The system follows a client-server architecture, where the React.js frontend communicates with the Node.js and Express.js backend using REST APIs. MongoDB serves as the NoSQL database for storing user information, complaint records, feedback, and other application data. Authentication and authorization are implemented using JSON Web Tokens (JWT), while bcrypt secures user passwords through encryption. Multer is used for uploading complaint and resolution images, and Nodemailer handles password reset and email notification functionalities.

The application is designed with a responsive user interface, ensuring compatibility across desktop, laptop, tablet, and mobile devices. By digitizing the complaint management process, the system improves transparency, reduces manual effort, enhances communication, and enables faster complaint resolution.

Technologies Used

- **Frontend:** React.js, HTML5, CSS3, Bootstrap, JavaScript
- **Backend:** Node.js, Express.js
- **Database:** MongoDB with Mongoose ODM
- **Authentication:** JWT, bcryptjs
- **File Upload:** Multer
- **Email Service:** Nodemailer
- **API Communication:** REST APIs using Axios
- **Development Tools:** Visual Studio Code, Postman, MongoDB Compass, Git, GitHub

2. Project Overview

Purpose:

The **Online Complaint Registration** is designed to provide a centralized and efficient platform for registering, managing, tracking, and resolving complaints. The primary objective of the system is to replace the traditional manual complaint handling process with a secure, transparent, and user-friendly web application. The system enables users to submit complaints online, monitor their complaint status, communicate with the responsible authorities, and provide feedback after the complaint is resolved. Administrators can efficiently manage users, approve agents, assign complaints, and monitor complaint progress, while agents can update complaint status and resolve complaints in a timely manner.

Key Features:

1. User Registration

Allows new users to create an account by providing their personal details with proper validation and secure password storage.

2. Secure Login

Enables registered users to securely access the system using their email and password through JWT-based authentication.

3. Forgot Password

Allows users to reset their password securely through an email-based password recovery mechanism.

4. Role-Based Access Control

Provides separate access and functionalities for Ordinary Users, Agents, and Admins based on their roles.

5. Complaint Registration

Users can submit complaints by entering complaint details and uploading supporting images.

6. Complaint Tracking

Allows users to monitor the current status of their complaints from submission until resolution.

7. Complaint Assignment

Enables administrators to assign complaints to available agents for timely resolution.

8. Agent Management

Allows administrators to create, approve, and manage agent accounts within the system.

9. Complaint Resolution

Agents can update complaint status, upload resolution images, and mark complaints as resolved.

10. Dashboard Analytics

Provides administrators with complaint statistics, charts, and summary reports for effective monitoring.

11. Feedback Management

Users can submit ratings and feedback after complaint resolution to improve service quality.

12. Profile Management

Allows users to view and update their profile information whenever required.

13. Image Upload

Supports uploading complaint evidence and resolution images using secure file handling.

14. Email Notifications

Sends automated emails for password reset requests and complaint assignment notifications.

15. Responsive Design

Ensures the application works smoothly across desktops, laptops, tablets, and mobile devices.

16. Secure Data Management

Stores user and complaint data securely using MongoDB with password encryption and JWT authorization.

3. Architecture

The **Online Complaint Registration** follows a three-tier client-server architecture developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js). The architecture separates the application into Presentation Layer (Frontend), Business Logic Layer (Backend), and Data Layer (Database), making the system modular, secure, scalable, and easy to maintain. The React.js frontend communicates with the Express.js backend through REST APIs, while MongoDB stores all application data.

3.1 Frontend Architecture (React.js)

The frontend of the application is developed using **React.js**, providing a responsive and interactive user interface. The application follows a component-based architecture where reusable components improve maintainability and reduce code duplication. Routing is managed using React Router, while Axios is used to communicate with the backend through REST APIs.

The frontend provides dedicated interfaces for the three user roles:

- **Ordinary User Dashboard**
- **Agent Dashboard**
- **Admin Dashboard**

Frontend Responsibilities

- User Registration and Login
- Forgot Password
- Complaint Registration
- Complaint Status Tracking
- Profile Management
- Complaint Image Upload
- Dashboard Visualization
- Agent Management
- Feedback Submission
- Responsive User Interface

Technologies Used

- React.js
- HTML5
- CSS3
- Bootstrap
- JavaScript
- Axios
- React Router DOM

3.2 Backend Architecture (Node.js & Express.js)

The backend is developed using **Node.js** with the **Express.js** framework. It follows the MVC (Model–View–Controller) architectural pattern to separate business logic, routing, and database operations.

The backend exposes RESTful APIs that process client requests, validate user input, perform authentication and authorization, manage complaint workflows, and communicate with MongoDB.

Backend Responsibilities

- User Authentication
- JWT Token Generation & Verification
- Complaint Management
- Complaint Assignment
- Agent Approval
- Complaint Status Update
- Feedback Management
- Profile Management
- Password Reset
- Email Notification
- Image Upload Management

Technologies Used

- Node.js
- Express.js
- JWT
- bcryptjs
- Multer
- Nodemailer
- Express Validator
- Helmet
- CORS

3.3 Database Architecture (MongoDB)

The application uses **MongoDB**, a NoSQL database, to store application data efficiently. MongoDB collections are managed using Mongoose ODM, which provides schema-based modeling, validation, and simplified database interactions.

The primary collections used in the system are:

- Users
- Complaints
- Feedback
- Chats

Database Responsibilities

- Store User Information
- Store Complaint Details
- Store Complaint Status
- Store Uploaded Image Paths
- Store Feedback Records
- Maintain Relationships Between Users and Complaints

Technologies Used

- MongoDB
- Mongoose ODM

4. Setup Instructions

Prerequisites:

- Node.js
- npm (Node Package Manager)
- Visual Studio Code
- MongoDB Atlas
- Git
- Postman
- Google Chrome / Microsoft Edge

Installation:

Step 1: Clone the Repository

Open the terminal and execute the following command:

```
git clone https://github.com/yourusername/online-complaint-registration.git
```

Step 2: Navigate to the Project Directory

cd online-complaint-registration

Step 3: Install Frontend Dependencies

Navigate to the client folder.

cd client

Install all required packages.

npm install

Step 4: Install Backend Dependencies

Open another terminal.

Navigate to the server folder.

cd server

Install backend packages.

npm install

Environment Variables

Create a **.env** file inside the **server** folder and configure the following environment variables.

PORT=5000

MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_jwt_secret_key

EMAIL_USER=your_email@gmail.com

EMAIL_PASS=your_email_app_password

CLIENT_URL=http://localhost:5173

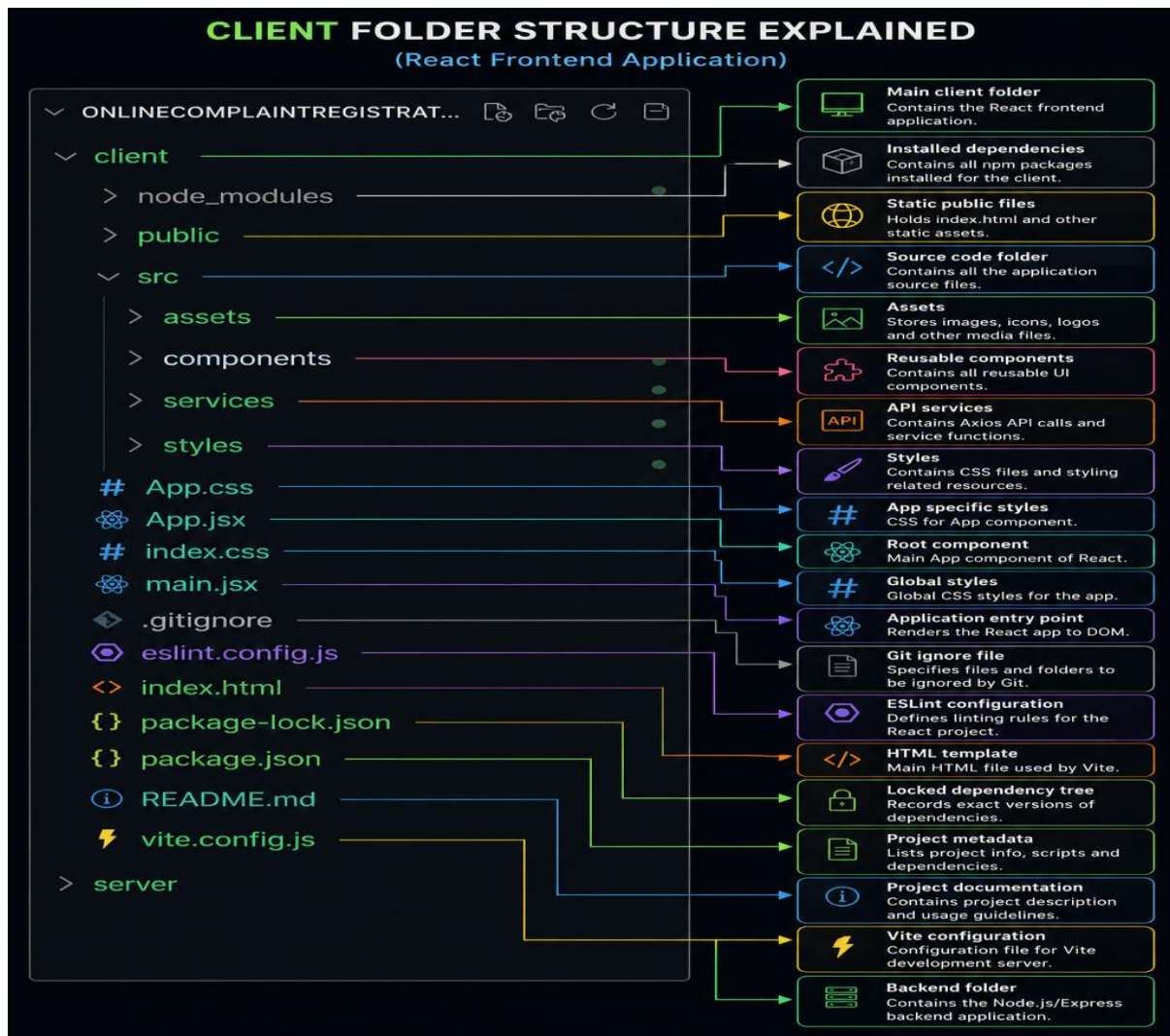
Note: Replace the placeholder values with your actual MongoDB connection string, JWT secret key, and Gmail App Password before running the application.

5. Folder Structure

The **Online Complaint Registration** follows a modular folder structure that separates the frontend and backend into independent modules. This organization improves code readability, maintainability, and scalability.

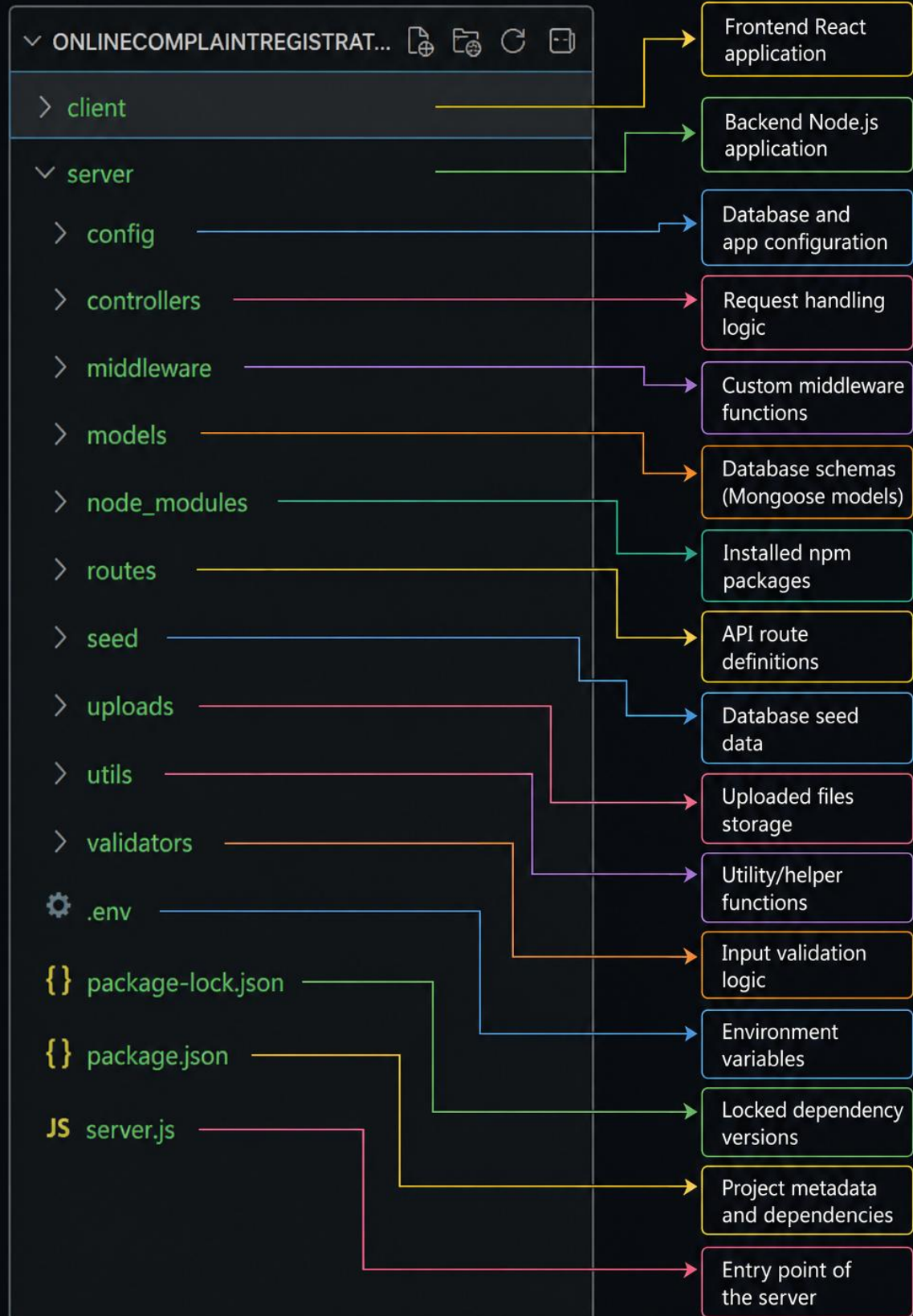
The project is divided into two main directories:

Client – Contains the React.js frontend application responsible for the user interface and user interactions.



Server – Contains the Node.js and Express.js backend responsible for business logic, REST APIs, authentication, database communication, and file management.

PROJECT FOLDER STRUCTURE EXPLAINED



6. Running the Application

After successfully installing all project dependencies and configuring the environment variables, the Online Complaint Registration and Management System can be executed by starting both the backend and frontend servers.

6.1 Start the Backend Server

Open a terminal and navigate to the server directory.

```
cd server
```

Start the backend server using the following command:

```
npm start
```

If the server starts successfully, the terminal displays:

```
MongoDB Connected Successfully
```

```
Server running on port 5000
```

The backend server will be available at:

```
http://localhost:5000
```

6.2 Start the Frontend Application

Open another terminal and navigate to the client directory.

```
cd client
```

Start the React application using Vite:

```
npm run dev
```

After successful execution, Vite displays:

```
VITE vX.X.X ready in XXX ms
```

→ Local: <http://localhost:5173/>

Open the following URL in your browser:

```
http://localhost:5173
```

7. API Documentation

The backend of the Online Complaint Registration and Management System exposes RESTful APIs developed using Node.js and Express.js. These APIs enable communication between the React frontend and the MongoDB database. Authentication is secured using JWT, and all protected APIs require a valid authorization token.

7.1 Authentication APIs

Method	API Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Authenticate user and generate JWT token
PUT	/api/auth/forgot-password	Send password reset email
GET	/api/auth/profile	Retrieve logged-in user's profile
PUT	/api/auth/profile	Update logged-in user's profile
POST	/api/auth/logout	Logout the current user
GET	/api/auth/users	Retrieve all registered users (Admin)
GET	/api/auth/statistics	Retrieve dashboard statistics (Admin)
POST	/api/auth/create-agent	Create a new agent account (Admin)
PUT	/api/auth/approve-agent/:id	Approve an agent account (Admin)

7.2 Complaint APIs

Method	API Endpoint	Description
POST	/api/complaints/create	Register a new complaint
GET	/api/complaints/my-complaints	Retrieve complaints submitted by the logged-in user
GET	/api/complaints/all	Retrieve all complaints (Admin)

Method	API Endpoint	Description
GET	/api/complaints/assigned	Retrieve complaints assigned to the logged-in agent
GET	/api/complaints/:id	Retrieve complaint details by ID
PUT	/api/complaints/update/:id	Update complaint details
PUT	/api/complaints/cancel/:id	Cancel a complaint
PUT	/api/complaints/reopen/:id	Reopen a cancelled complaint
PUT	/api/complaints/assign/:id	Assign a complaint to an agent (Admin)
PUT	/api/complaints/status/:id	Update complaint status and upload resolution image (Agent)

7.3 Feedback APIs

Method	API Endpoint	Description
POST	/api/feedback/submit	Submit feedback after complaint resolution
GET	/api/feedback/all	Retrieve all user feedback (Admin)

8. Authentication

The Online Complaint Registration uses JSON Web Token (JWT) for secure authentication and Role-Based Access Control (RBAC) for authorization. User passwords are encrypted using bcryptjs before being stored in MongoDB. After successful login, a JWT token is generated and stored in the browser's local storage. This token is included in the Authorization header to access protected REST APIs.

8.1 Authentication Workflow

User Login

|



Email & Password Verification

|



JWT Token Generated



Token Stored in Local Storage



Token Sent with REST API Requests



Protected Resources Accessed

8.2 Role-Based Authorization

Role	Permissions
Ordinary User	Register, Login, Manage Profile, Register & Track Complaints, Submit Feedback
Agent	View Assigned Complaints, Update Complaint Status, Upload Resolution Images
Admin	Manage Users & Agents, Assign Complaints, View Dashboard & Feedback

8.3 Security Features

- JWT-based Authentication
- Password Encryption using **bcryptjs**
- Role-Based Access Control (RBAC)
- Protected Routes using Authentication Middleware
- Request Validation using Express Validator
- Secure Configuration using Environment Variables

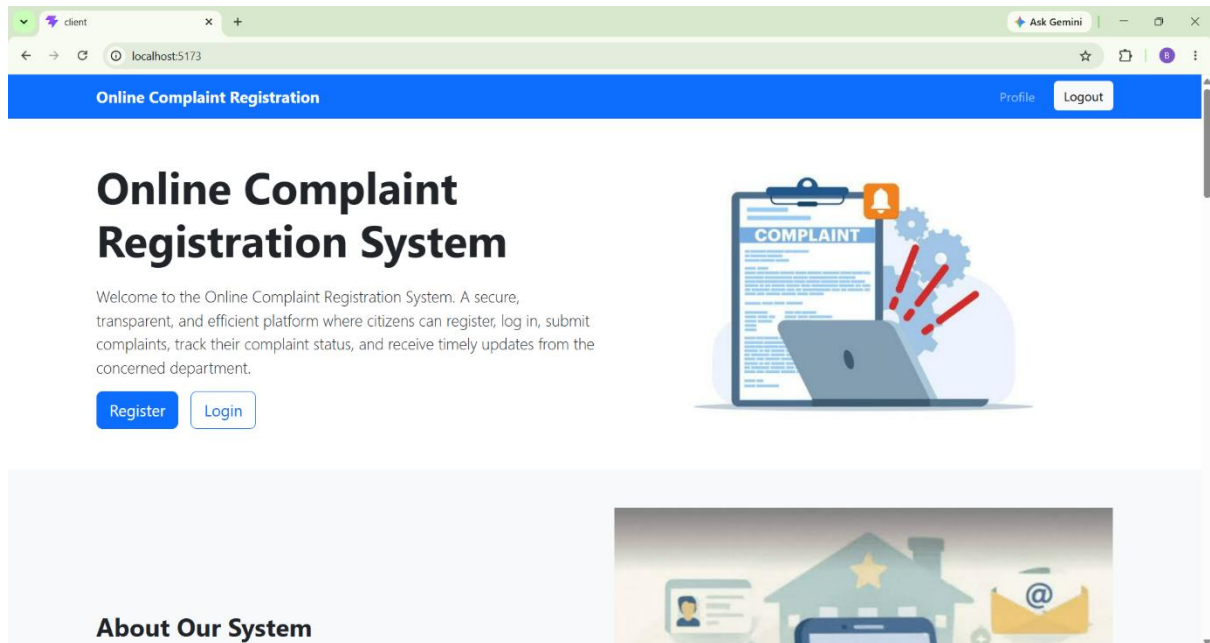
9. User Interface

The **Online Complaint Registration** provides a simple, responsive, and user-friendly interface developed using React.js and Bootstrap. Separate dashboards are provided for

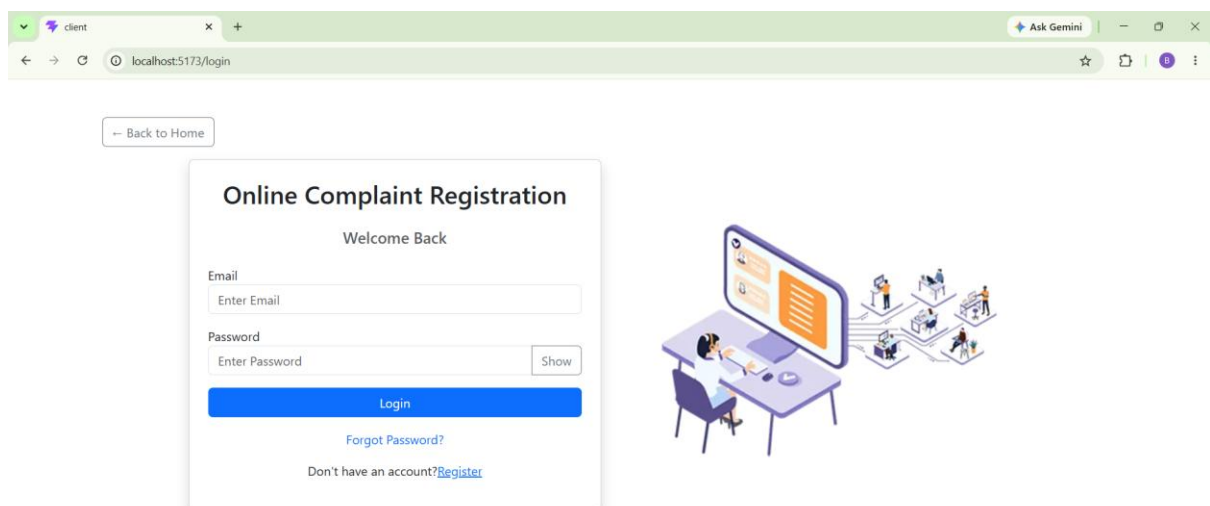
Ordinary Users, Agents, and Administrators, ensuring easy navigation and efficient complaint management.

Screenshots:

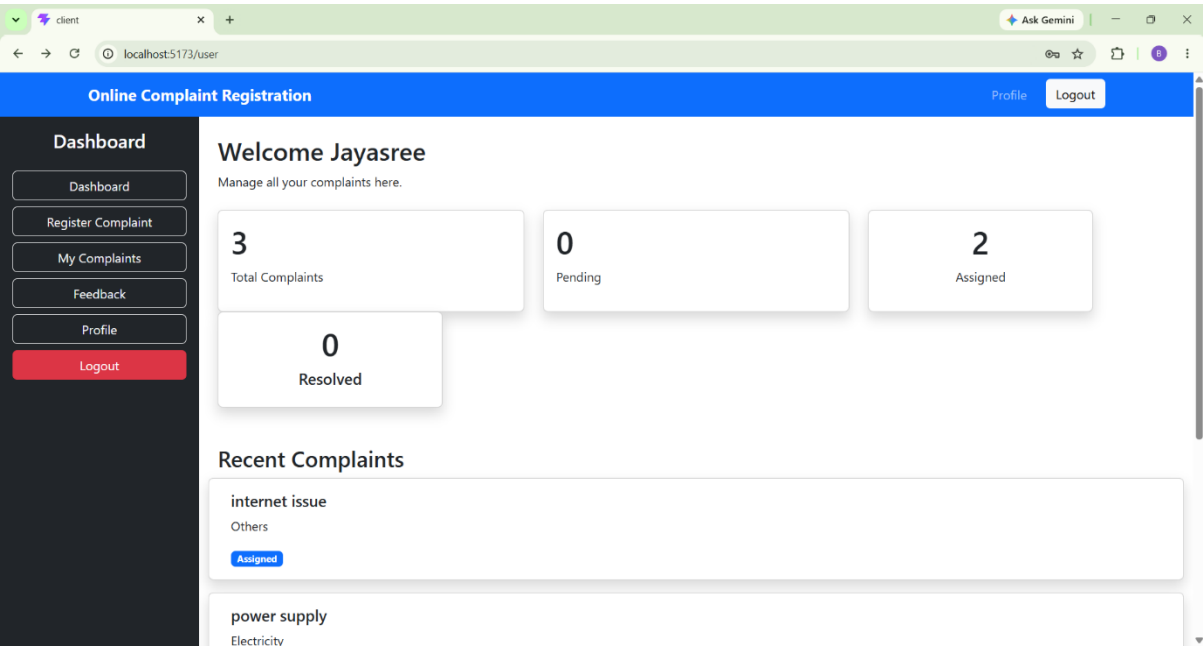
Landing Page



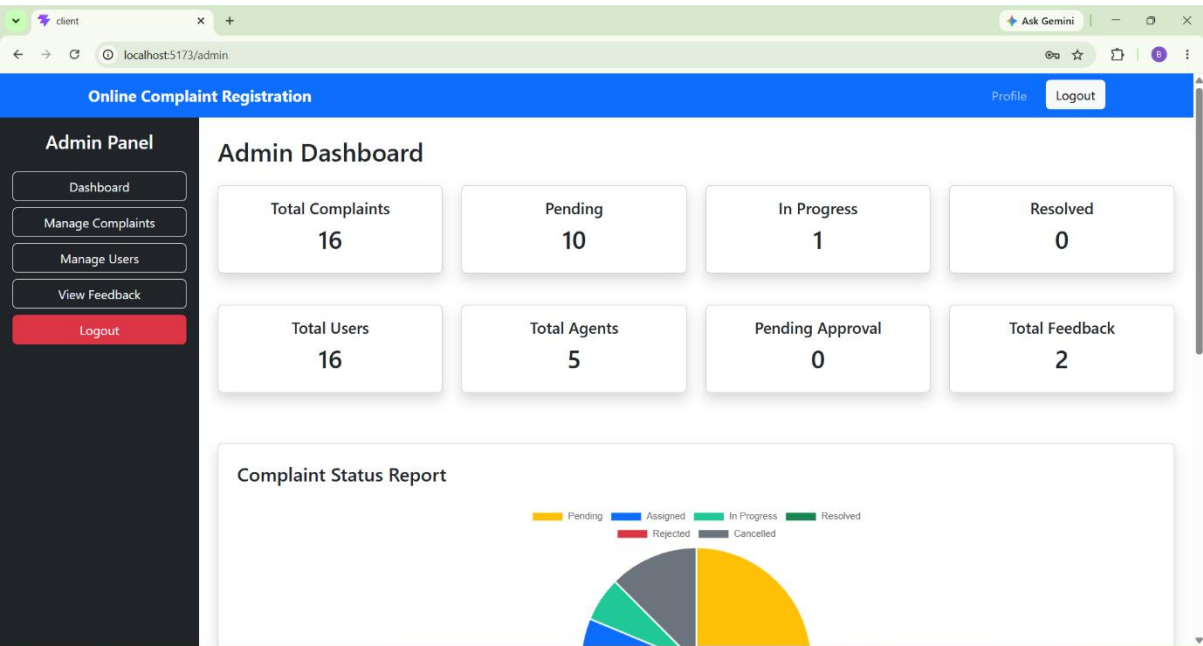
Login Page:



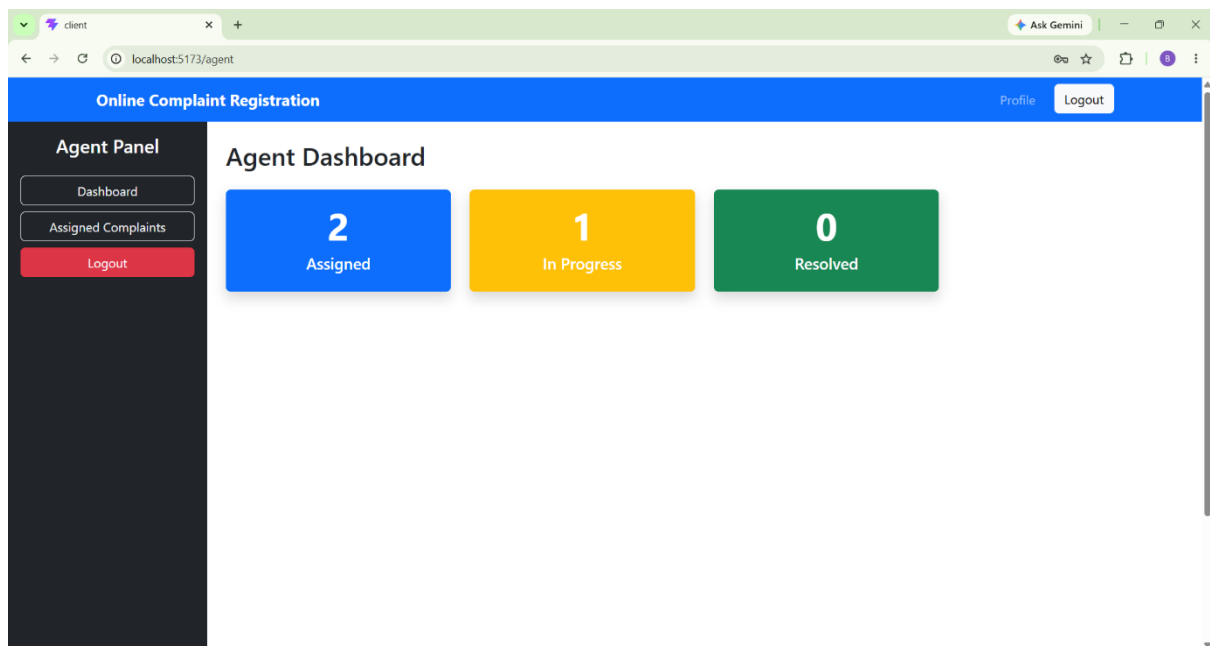
User Dashboard:



Admin Dashboard:



Agent Dashboard:



10. Testing

The **Online Complaint Registration** was tested to verify that all functionalities work correctly and meet the project requirements. Testing was carried out on the frontend, backend, database, and REST APIs to ensure reliable system performance.

Testing Tools

Tool	Purpose
Postman	REST API Testing
Google Chrome	Functional & UI Testing
MongoDB Compass	Database Verification
Visual Studio Code	Debugging

Modules Tested:

- User Registration & Login
- Forgot Password
- Complaint Registration & Tracking
- Complaint Assignment

- Complaint Status Update
- Profile Management
- Feedback Submission
- Dashboard Analytics

Testing Result:

All modules were tested successfully. The application functioned as expected, REST APIs responded correctly, authentication worked securely, and the system performed efficiently across different devices.

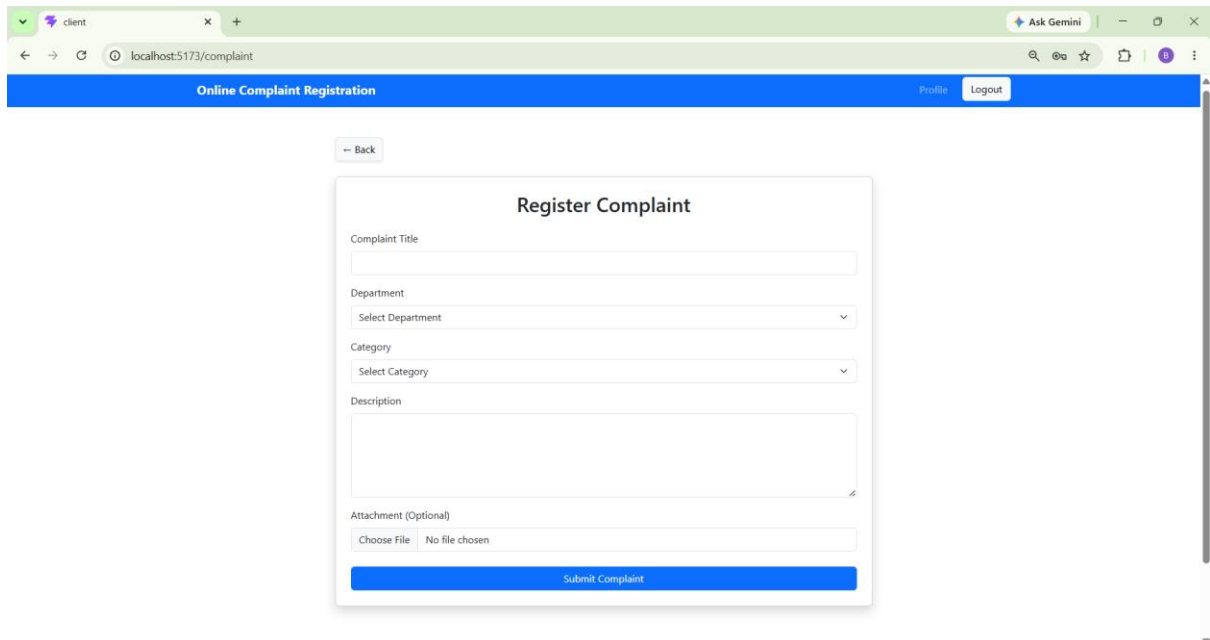
11. Screenshots or Demo

Demo Link:

https://drive.google.com/file/d/1xpFCIIzqJvkBHhfcYxkw_vTcWLsurJ9E/view?usp=sharing

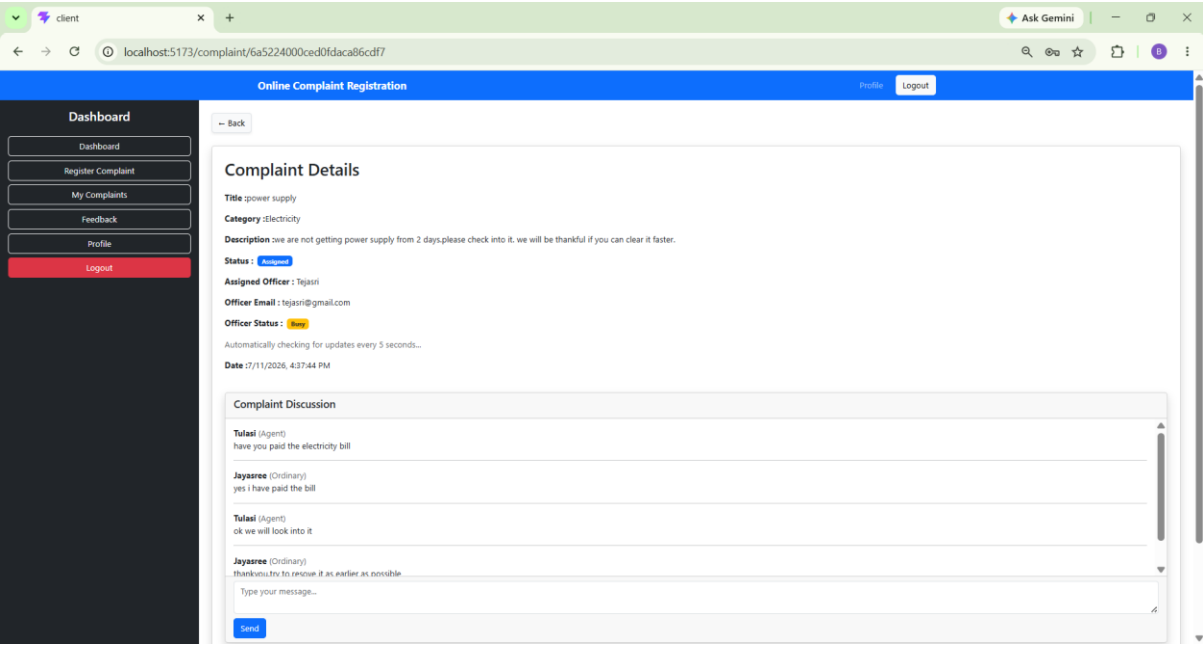
Screenshots:

Complaint Registration:

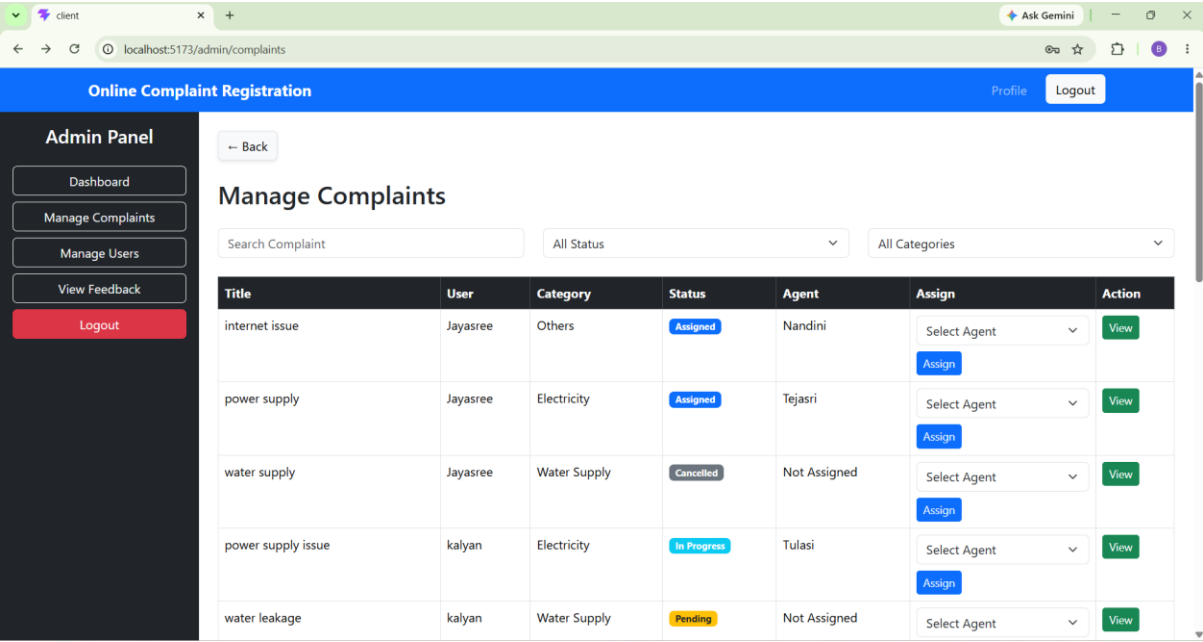


The screenshot shows a web browser window with the URL `localhost:5173/complaint`. The page has a blue header bar with the text "Online Complaint Registration" on the left and "Profile" and "Logout" links on the right. Below the header, there is a "Back" button. The main content is a "Register Complaint" form. The form contains the following fields: "Complaint Title" (text input), "Department" (dropdown menu with "Select Department" text), "Category" (dropdown menu with "Select Category" text), "Description" (text area), and "Attachment (Optional)" (file upload button showing "Choose File" and "No file chosen"). At the bottom of the form is a blue "Submit Complaint" button.

Complaint Tracking:



Complaint Assignment:



Complaint Status Update:

client

Ask Gemini

localhost:5173/agent/complaints

Online Complaint Registration

ProfileLogout

Agent Panel

Dashboard

Assigned Complaints

Logout

Back

Assigned Complaints

power supply issue

There is no power in our area from morning.kids and elders are feeling uncomfortable.please look in to the issue. thank you.

Category: Electricity

User: kalyan

Email: kp@gmail.com

Status

In Progress

Action Notes

Update Complaint

Complaint Discussion

Tulasi (Agent)

have you paid current bill

kalyan (Ordinary)

yes

Type your message...

Feedback Submission:

client

Ask Gemini

localhost:5173/feedback

Online Complaint Registration

ProfileLogout

Dashboard

Dashboard

Register Complaint

My Complaints

Feedback

Profile

Logout

Back

Submit Feedback

Complaint

Select Complaint

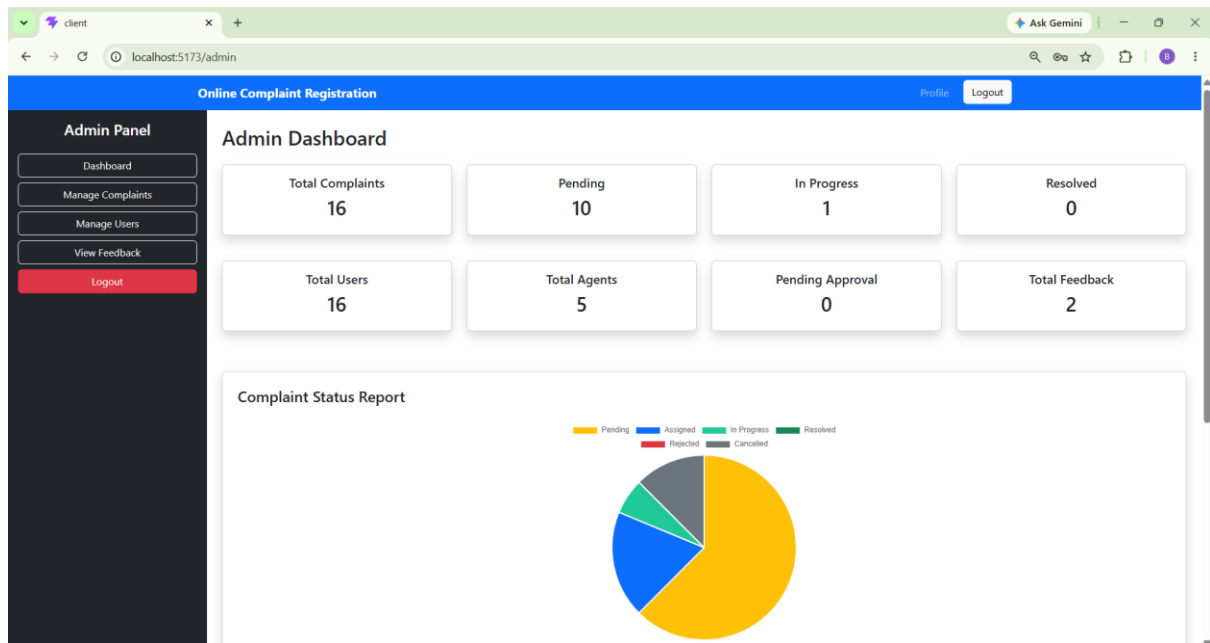
Rating

Select Rating

Message

Submit Feedback

Dashboard Analytics:



12. Known Issues

The current version of the **Online Complaint Registration** functions as expected. However, a few limitations have been identified that can be improved in future versions.

- Email notifications depend on a valid internet connection and SMTP service.
- The application is currently designed for local deployment.
- Advanced search and filtering options for complaints are limited.

13. Future Enhancements

The following enhancements can be implemented in future versions to improve the functionality and user experience of the system:

- Deploy the application on a cloud platform for public access.
- Implement multi-language support.
- Add SMS and push notification services for complaint updates.
- Introduce advanced search and filtering options.
- Generate downloadable complaint reports (PDF/Excel).
- Integrate AI-based complaint categorization and priority prediction.

- Develop a mobile application for Android and iOS.
- Implement real-time chat and notifications for faster communication.